Thetawave.ai

💻 代理人模式 – 核心概念與應用解析

Generated by Thetawave.ai

🛡️ 代理人模式 (Proxy Pattern) 簡介與應用場景

> **代理人模式**是一種控制對某物件存取的設計模式。當需要在不修改既有程式碼的前提下，延遲實體化昂貴物件或限制使用者權限時，代理人模式就派上用場。

---

1. 需求背景與系統架構

- **專案**：開發一款簡易的記帳系統（Expense Tracking System）
- **功能**：
  - 新增帳目 (Add Expense)：輸入描述與金額，系統儲存帳目並記錄建立時間
  - 編輯帳目 (Edit Expense)：可修改帳目描述與金額
  - 匯出CSV檔案 (Export CSV File)：匯出所有帳目為CSV格式
- **介面**：使用命令行介面 (Command Line Interface, CLI)，暫無GUI
- **架構重點**：
  - 定義 Expense Tracking System 介面，包含四種操作：取得所有帳目 (Get Expenses)、新增、修改、匯出CSV
  - 帳目類別 Expense 具備 ID（唯一識別）、描述、金額（BigDecimal類型以避免浮點誤差）、建立時間 (LocalDateTime)
  - 實作類別 Super Expense Tracking System，負責管理帳目資料，並以 JSON 格式存取資料檔案 (expenses.json)

Thetawave.ai

2. 物件導向設計與問題點

角色	職責與特點
Expense Tracking (CLI)	與使用者互動，將使用者需求委派給 Expense Tracking System
Expense Tracking System (介面)	定義記帳系統操作規範
Expense	帳目資料類別，ID唯一且由UUID亂數產生
Super Expense Tracking System	實作記帳系統，管理帳目資料，載入 JSON 檔案

- **問題一**：Super Expense Tracking System 載入大量帳目資料，啟動時耗時且浪費記憶體，若使用者快速關閉程式，資源浪費嚴重
- **問題二**：不允許修改小華撰寫的 Super Expense Tracking System 類別（即不改既有實作）
- **問題三**：不希望修改 CLI 與介面（即不變動既有 Client 端程式碼）
- **問題四**：希望延遲實體化昂貴物件（Super Expense Tracking System），直到真的需要使用時才載入資料

---

3. 代理人模式的核心解決方案

3.1 問題定義

> 如何在不修改既有程式碼的情況下，控制 Client 對昂貴物件的存取，並達成延遲載入？

3.2 方案概述

- 引入代理人類別（Proxy），實作與 Super Expense Tracking System 同樣的介面 Expense Tracking System
- Client (Expense Tracking CLI) 不直接依賴 Super Expense Tracking System，而是依賴代理人物件
- 代理人內部持有 Super Expense Tracking System 的參考，但初始為 null

- 每次執行代理人方法時，先檢查 Super Expense Tracking System 是否已實體化，若尚未則延遲載入（Lazy Initialization）
- 代理人透過委派呼叫實際的 Super Expense Tracking System 方法，達成功能效果

3.3 延遲載入 (Lazy Initialization)

> 延遲載入指的是「當真正需要使用該物件時，才實體化並載入相關資源」，避免不必要的系統負擔。

---

4. 代理人模式實作細節與類別結構

類別名稱	功能說明
Virtual Super Expense Tracking System Proxy	代理人類別，實作 Expense Tracking System 介面，負責延遲實體化 Super 物件並委派方法呼叫
Super Expense Tracking System	實際的記帳系統類別，管理帳目資料及檔案
Expense Tracking (CLI)	依賴 Expense Tracking System 介面，透過依賴注入使用代理人物件

4.1 代理人類別 Virtual Proxy 核心方法示意

```
public class VirtualSuperExpenseTrackingSystemProxy implements
ExpenseTrackingSystem {
    private SuperExpenseTrackingSystem superSystem = null;

    private void lazyInit() {
        if (superSystem == null) {
            superSystem = new SuperExpenseTrackingSystem();
        }
    }

    @Override
    public void addExpense(Expense expense) {
        lazyInit();
        superSystem.addExpense(expense);
    }
```

Thetawave.ai

```
    // 其他方法同理，先呼叫 lazyInit() 再委派
}
```

4.2 使用者體驗改進

- 啟動 CLI 時不會立刻載入大量帳目資料
- 直到使用者執行新增、編輯等操作時，才開始載入資料
- 節省系統資源，提升啟動速度與效能

---

5. 代理人模式應對新需求：功能限制與版本控管

5.1 新需求

- 系統區分「免費版本（Trial Version）」與「付費版本（Premium Version）」
- 只有付費版本可使用「匯出CSV檔案」功能
- 免費版本中，嘗試使用匯出功能需**中斷存取並丟出例外**

5.2 解決方案

- 在已有 Virtual Proxy 的基礎上，新增一個**免費版本代理人類別**：Trial Version Proxy
- Trial Proxy 繼承 Virtual Proxy，覆寫匯出CSV方法，直接丟出不支援操作的例外
- 在免費版本中，將 Trial Proxy 注入至 CLI 端，達成權限限制

5.3 實作示意

```
public class TrialVersionSuperExpenseTrackingSystemProxy extends
VirtualSuperExpenseTrackingSystemProxy {
    @Override
    public void exportCSV(String filename) {
        throw new UnsupportedOperationException("CSV exporting is not
supported in the trial version.");
```

```
    }
}
```

Thetawave.ai

---

6. 代理人模式（Proxy Pattern）核心總結

項目	說明
Context	Client 依賴某介面，且有類別實作此介面
Force 1	Client 必須依賴實作介面的物件，且需依賴注入
Force 2	希望控制 Client 對實體物件的存取（延遲載入、權限控管等）
Force 3	不想修改既有程式碼
Problem	如何在不改變既有程式碼前提下，控制 Client 對實體物件的存取
Solution (Form)	建立 Proxy 類別實作相同介面，並代理實體物件，Client 只與 Proxy 互動
Resulting Context	- Proxy 可在 Client 存取實體物件前加入額外行為（權限控管、日誌、快取等） - 實體物件可延遲載入 - 實體物件可在不同記憶體空間（例如遠端代理 Remote Proxy）

---

7. 代理人模式的分類與變形

Proxy 類型	主要用途與特點
Virtual Proxy	延遲載入昂貴物件，直到真正需要時才實體化
Protection Proxy	控制對實體物件的存取權限，防止未授權操作
Remote Proxy	代理遠端物件，讓 Client 感覺物件就在本地，但實際透過通訊協定存取

Thetawave.ai

8. 代理人模式擴充問題與進階思考

- 若 Proxy 不直接關聯實體類別，而只關聯介面（Subject Interface）：
  - 雖然 Proxy 可代理多種實作類別，但就無法知道該如何實體化實際物件
  - 延遲載入功能難以實作，因為不清楚實體類別的具體型態
- 解決方案通常需借助其他**創建型模式 (Creational Pattern)**，如工廠模式（Factory Pattern）來動態產生實體物件

---

9. 代理人模式的比喻與理解

- 將代理人 (Proxy) 想像成「經紀人」
- 實體物件 (RealSubject) 是「大牌藝人」
- Client 想與藝人互動，但藝人忙碌且昂貴，經紀人先行代理出席活動，直到真正需要時才叫藝人本人出場
- 此比喻說明延遲載入與權限控管的實際意義及運作方式

代理人模式透過讓代理物件控制實體物件的存取，不僅實現了**延遲載入**與**權限控管**，更避免改動既有程式結構，促使系統維護與擴充更為方便，並且能因應不同版本需求靈活調整使用權限。代理物件的存在使得系統架構更具彈性，並且能有效管理昂貴資源的使用時機與範圍。